

// Jivops

Kubeflow en Producción

Guía completa — 30 lecciones

Arquitectura · Notebooks · Pipelines · Entrenamiento distribuido · Serving y multi-tenancy · Operación avanzada

Contenido

Módulo 1: Fundamentos y arquitectura

- Lección 01 — Qué es Kubeflow y su propósito como plataforma de ML en Kubernetes
- Lección 02 — Arquitectura de componentes de Kubeflow
- Lección 03 — Instalación de Kubeflow (manifests, distribuciones)
- Lección 04 — Kubeflow Central Dashboard
- Lección 05 — Kubeflow vs plataformas ML gestionadas (SageMaker, Vertex AI)

Módulo 2: Notebooks y desarrollo

- Lección 06 — Kubeflow Notebooks — servidores Jupyter en el cluster
- Lección 07 — Perfiles (Profiles) y namespaces de usuario
- Lección 08 — Volúmenes persistentes para notebooks
- Lección 09 — Imágenes personalizadas de notebook
- Lección 10 — Recursos y cuotas para notebooks (CPU/GPU)

Módulo 3: Pipelines

- Lección 11 — Kubeflow Pipelines — componentes y arquitectura interna
- Lección 12 — SDK de KFP — definir un pipeline en Python
- Lección 13 — Compilación y ejecución de pipelines
- Lección 14 — Parámetros y artefactos entre componentes
- Lección 15 — Recurring runs y scheduling de pipelines

Módulo 4: Entrenamiento distribuido

- Lección 16 — Training Operator — TFJob, PyTorchJob
- Lección 17 — Entrenamiento distribuido con PyTorchJob
- Lección 18 — Katib — ajuste automático de hiperparámetros
- Lección 19 — Algoritmos de búsqueda en Katib
- Lección 20 — Experiment y Trial en Katib

Módulo 5: Serving y multi-tenancy

- Lección 21 — KServe dentro de Kubeflow
- Lección 22 — InferenceService — despliegue de modelos
- Lección 23 — Multi-tenancy en Kubeflow — aislamiento entre equipos
- Lección 24 — RBAC e Istio AuthorizationPolicy en Kubeflow
- Lección 25 — Integración con Model Registry

Módulo 6: Operación avanzada

Lección 26 — Metadata y linaje con ML Metadata (MLMD)

Lección 27 — Monitoreo de Kubeflow con Prometheus

Lección 28 — Actualización de Kubeflow sin downtime

Lección 29 — Troubleshooting de pipelines fallidos

Lección 30 — Proyecto final: plataforma Kubeflow completa lista para producción

Módulo 1: Fundamentos y arquitectura

Lección 01 — Qué es Kubeflow y su propósito como plataforma de ML en Kubernetes

Proveer un conjunto integrado de herramientas para todo el ciclo de vida de machine learning (desarrollo, entrenamiento, tuning, despliegue) aprovechando la infraestructura de Kubernetes existente.

Un equipo que ya opera Kubernetes puede aprovechar directamente su capacidad de escalado, scheduling y gestión de recursos existente, sin tener que aprender ni mantener un sistema de infraestructura completamente distinto y separado solo para las cargas de trabajo de ML.

Lección 02 — Arquitectura de componentes de Kubeflow

Cada componente es relativamente independiente y puede instalarse o usarse por separado según la necesidad, en vez de requerir obligatoriamente la instalación completa de todos a la vez.

Un equipo puede empezar adoptando solo el componente que resuelve su necesidad más inmediata (como Pipelines para orquestación), sin verse obligado a desplegar y mantener la plataforma completa desde el primer día, incorporando otros componentes gradualmente según surjan necesidades reales.

Lección 03 — Instalación de Kubeflow (manifests, distribuciones)

Una distribución de un proveedor cloud suele integrar Kubeflow con servicios nativos de esa plataforma (almacenamiento, IAM, redes), mientras los manifests oficiales dan una instalación más genérica y portable entre entornos.

Los manifests oficiales portables permiten una configuración más consistente entre distintos proveedores cloud donde la organización ya opera, evitando quedar atada a integraciones específicas de un único proveedor que dificultarían replicar la misma instalación en otro entorno.

Lección 04 — Kubeflow Central Dashboard

Ofrece un punto de entrada visual unificado desde donde navegar hacia los distintos componentes instalados (Notebooks, Pipelines, Experiments), sin tener que acceder a cada uno por separado.

En vez de tener que aprender de memoria o buscar la URL específica de cada componente distinto, el nuevo miembro del equipo puede explorar visualmente qué herramientas están disponibles desde un único punto de entrada central, reduciendo la fricción inicial de adopción.

Lección 05 — Kubeflow vs plataformas ML gestionadas (SageMaker, Vertex AI)

Mayor control y portabilidad al ser una plataforma de código abierto que se ejecuta sobre la propia infraestructura de Kubernetes, sin atarse a un único proveedor cloud específico.

Al autoalojar Kubeflow sobre su propia infraestructura de Kubernetes, la organización mantiene control directo y verificable sobre exactamente dónde residen sus datos y modelos, algo importante para cumplir requisitos regulatorios específicos que una plataforma gestionada externa podría no satisfacer completamente.

Módulo 2: Notebooks y desarrollo

Lección 06 — Kubeflow Notebooks — servidores Jupyter en el cluster

Despliega un Pod dedicado con el entorno Jupyter configurado, con los recursos de cómputo (CPU/GPU) y almacenamiento persistente solicitados, aislado dentro del namespace del usuario.

El cluster puede tener GPUs u otros recursos especializados de cómputo que la laptop personal del usuario normalmente no tendría disponibles, permitiendo trabajar con cargas de entrenamiento más exigentes directamente desde el notebook remoto sin esa limitación local.

Lección 07 — Perfiles (Profiles) y namespaces de usuario

Un namespace de Kubernetes dedicado para ese usuario, junto con los permisos RBAC necesarios para que pueda operar dentro de él de forma aislada de otros usuarios.

Al tener cada usuario sus propios recursos contenidos dentro de un namespace separado y aislado, un notebook o pipeline mal configurado de un usuario no tiene forma directa de acceder o interferir con los recursos de otro usuario que vive en un namespace completamente distinto.

Lección 08 — Volúmenes persistentes para notebooks

Sin un volumen persistente, todo el trabajo (código, datos, modelos) guardado dentro del notebook se perdería si el Pod se reinicia o se elimina por cualquier motivo.

Un reinicio del Pod (por mantenimiento del nodo, actualización, o cualquier otro motivo operativo normal) es un evento que puede ocurrir en cualquier momento; sin un volumen persistente respaldando ese trabajo, todo el progreso acumulado durante días de experimentación se perdería sin ninguna forma de recuperación.

Lección 09 — Imágenes personalizadas de notebook

Para incluir librerías, dependencias o herramientas específicas del proyecto que no vienen preinstaladas en las imágenes base genéricas, evitando tener que instalarlas manualmente cada vez que se crea un notebook nuevo.

Con una imagen ya preparada, cada notebook nuevo arranca con el entorno completo listo de inmediato, sin que cada persona del equipo tenga que repetir manualmente el mismo proceso de instalación de dependencias cada vez que necesita un notebook nuevo.

Lección 10 — Recursos y cuotas para notebooks (CPU/GPU)

Evita que los notebooks de un equipo consuman una cantidad desproporcionada de recursos compartidos del cluster (CPU, memoria, GPU), afectando la disponibilidad para otros equipos.

En un cluster compartido con un número limitado de GPUs disponibles, sin ninguna cuota que lo impida, un equipo con varios notebooks de entrenamiento intensivo corriendo simultáneamente podría consumir accidentalmente toda la capacidad disponible, dejando a otros equipos sin ninguna GPU libre para su propio trabajo.

Módulo 3: Pipelines

Lección 11 — Kubeflow Pipelines — componentes y arquitectura interna

Una unidad de trabajo autocontenida (empaquetada como contenedor) con entradas y salidas bien definidas, que realiza un paso específico del flujo de ML.

Como cada componente corre en su propio contenedor con sus propias dependencias instaladas, un paso de preprocesamiento que necesita una versión específica de una librería no entra en conflicto con un paso de entrenamiento que necesita una versión distinta de esa misma librería, ya que cada uno vive en su entorno aislado.

Lección 12 — SDK de KFP — definir un pipeline en Python

Describir el flujo de componentes y sus dependencias usando decoradores y funciones Python, que luego se traducen a una definición ejecutable del pipeline en el cluster.

El científico de datos puede aprovechar directamente su conocimiento existente de Python para definir la lógica y el flujo del pipeline, sin necesitar aprender primero la sintaxis específica de definición de recursos de Kubernetes que requeriría escribir el YAML manualmente.

Lección 13 — Compilación y ejecución de pipelines

Un archivo de definición del pipeline (en formato YAML o similar) que describe la estructura completa del flujo, listo para subirse y ejecutarse en Kubeflow Pipelines.

El archivo compilado es una representación autocontenida y portable del pipeline que puede versionarse, compartirse o subirse directamente al sistema sin depender de tener el entorno de desarrollo Python original configurado, facilitando su distribución y ejecución reproducible.

Lección 14 — Parámetros y artefactos entre componentes

Un parámetro suele ser un valor simple (como un número o string) pasado directamente, mientras un artefacto representa datos más grandes o complejos (como un archivo o dataset) referenciado mediante su ubicación de almacenamiento.

Un parámetro está pensado para valores pequeños y simples; intentar pasar un dataset completo de varios gigabytes de esa forma sería ineficiente, mientras un artefacto simplemente referencia dónde está almacenado ese dataset, evitando mover físicamente datos pesados de forma innecesaria entre cada paso.

Lección 15 — Recurring runs y scheduling de pipelines

Ejecuta automáticamente el pipeline según un horario definido (por ejemplo, diariamente), sin necesitar que alguien lo dispare manualmente cada vez.

Una tarea manual repetitiva que depende de que alguien la recuerde ejecutar periódicamente es propensa al olvido con el tiempo; automatizar esa ejecución mediante un recurring run garantiza que el reentrenamiento ocurra de forma consistente según el horario definido, sin depender de la memoria de una persona.

Módulo 4: Entrenamiento distribuido

Lección 16 — Training Operator — TFJob, PyTorchJob

Coordina la creación y configuración de los Pods de tipo master y worker necesarios para el entrenamiento distribuido, incluyendo las variables de entorno de comunicación entre ellos.

Configurar manualmente la comunicación de red correcta entre múltiples Pods distribuidos (direcciones, puertos, roles de cada uno) es un proceso propenso a errores y tedioso; el Training Operator automatiza esa coordinación completa, reduciendo significativamente la complejidad y el riesgo de configuración incorrecta.

Lección 17 — Entrenamiento distribuido con PyTorchJob

Reducir el tiempo total de entrenamiento al paralelizar el cómputo entre múltiples nodos, especialmente valioso para modelos grandes o datasets masivos.

Si el modelo o el dataset completo excede la memoria disponible de una única máquina, simplemente no cabría para entrenarse ahí; distribuir el entrenamiento entre múltiples nodos permite dividir esa carga de memoria y cómputo entre varias máquinas trabajando en conjunto.

Lección 18 — Katib — ajuste automático de hiperparámetros

Lanza y coordina múltiples entrenamientos (Trials) con distintas combinaciones de hiperparámetros, recolectando las métricas de cada uno para identificar la combinación con mejor desempeño.

Probar manualmente docenas o cientos de combinaciones distintas, anotando cuidadosamente los resultados de cada una, sería extremadamente lento y propenso a errores de registro; Katib automatiza esa ejecución y comparación a mayor escala, sin intervención manual repetitiva.

Lección 19 — Algoritmos de búsqueda en Katib

La búsqueda bayesiana usa los resultados de Trials previos para decidir de forma más inteligente qué combinación probar a continuación, requiriendo típicamente menos Trials totales para encontrar una buena combinación.

El número total de combinaciones a probar en grid search crece multiplicativamente con cada nuevo hiperparámetro añadido a la búsqueda (una explosión combinatoria), haciendo que el costo total de ejecutar todos esos Trials se vuelva rápidamente impráctico a medida que se exploran más dimensiones.

Lección 20 — Experiment y Trial en Katib

Un Experiment define el espacio de búsqueda y el objetivo general de la optimización, mientras cada Trial es una ejecución individual concreta dentro de ese Experiment con una combinación específica de hiperparámetros.

Revisar los resultados de todos los Trials (no solo el mejor) permite observar patrones más amplios, como qué rangos de un hiperparámetro específico tienden a dar mejores resultados en general, información valiosa más allá de simplemente identificar la combinación ganadora individual.

Módulo 5: Serving y multi-tenancy

Lección 21 — KServe dentro de Kubeflow

Provee la capa estandarizada para desplegar modelos entrenados (posiblemente generados por un pipeline o Katib) como endpoints de inferencia servidos en el cluster.

Al estar integrado dentro del mismo ecosistema, el modelo producido por un pipeline de entrenamiento (o el mejor Trial de un experimento de Katib) puede desplegarse directamente vía KServe con menos pasos manuales de transferencia entre sistemas separados, dando un flujo más continuo de principio a fin.

Lección 22 — InferenceService — despliegue de modelos

La ubicación del artefacto del modelo entrenado, el runtime de predicción a usar, y la configuración de recursos y autoescalado para ese endpoint de inferencia específico.

Si el modelo tiene uso esporádico (no constante), mantenerlo siempre activo con recursos reservados desperdiciaría capacidad durante los períodos sin tráfico; escalar automáticamente hasta cero libera esos recursos cuando no se necesitan, reduciendo el costo real de operar ese modelo específico.

Lección 23 — Multi-tenancy en Kubeflow — aislamiento entre equipos

Namespaces separados por Profile, RBAC específico por usuario/equipo, y políticas de red o autorización de Istio que restringen la comunicación entre namespaces de distintos equipos.

Un namespace por sí solo organiza recursos lógicamente, pero por defecto no impide necesariamente que el tráfico de red fluya entre namespaces distintos; añadir políticas explícitas de autorización de Istio restringe activamente esa comunicación, dando una capa de aislamiento técnico adicional más robusta que solo la separación lógica de namespaces.

Lección 24 — RBAC e Istio AuthorizationPolicy en Kubeflow

Qué identidades (usuarios o servicios) tienen permitido realizar peticiones hacia los servicios dentro de ese namespace, a nivel de la malla de servicios de Istio.

El RBAC nativo de Kubernetes controla principalmente el acceso a la API de Kubernetes (crear, modificar recursos), mientras las políticas de Istio controlan directamente el tráfico de red real entre servicios que se comunican entre sí, dando una capa complementaria de protección a un nivel distinto y relevante para el tráfico entre componentes de Kubeflow.

Lección 25 — Integración con Model Registry

Permite registrar automáticamente el modelo resultante con su versión y metadata, facilitando su descubrimiento y promoción controlada hacia un InferenceService de producción posteriormente.

Si el registro dependiera de que alguien lo hiciera manualmente cada vez, sería fácil olvidar registrar alguna ejecución específica del pipeline; automatizarlo como parte del propio flujo garantiza que cada modelo producido quede documentado de forma consistente y trazable a su origen exacto.

Módulo 6: Operación avanzada

Lección 26 — Metadata y linaje con ML Metadata (MLMD)

Las relaciones entre artefactos, ejecuciones y contextos del pipeline, permitiendo reconstruir el linaje completo de cómo se produjo un modelo específico a partir de qué datos y parámetros.

Sin esa trazabilidad completa, sería muy difícil determinar retroactivamente exactamente qué combinación específica de datos, código y parámetros produjo ese modelo particular con el comportamiento problemático; el linaje de MLMD permite reconstruir ese origen exacto para investigar la causa raíz real.

Lección 27 — Monitoreo de Kubeflow con Prometheus

El estado de ejecución de pipelines (éxito/fallo), duración de Trials de Katib, y latencia/throughput de los InferenceServices activos.

Un pipeline de reentrenamiento podría fallar silenciosamente (por ejemplo, por un problema de datos) sin que eso se refleje necesariamente en métricas genéricas de CPU o memoria del cluster; monitorear específicamente el estado de éxito/fallo de cada ejecución del pipeline es lo que realmente revelaría ese problema específico de ML.

Lección 28 — Actualización de Kubeflow sin downtime

La compatibilidad entre la nueva versión de Kubeflow y la versión de Kubernetes del cluster subyacente, además de revisar cambios incompatibles en los CRDs de los componentes usados activamente.

Si una nueva versión de Kubeflow cambiara la estructura esperada de un CRD específico, los pipelines existentes escritos y compilados contra la versión anterior de esa estructura podrían dejar de ser compatibles, requiriendo ajustarlos antes de que la actualización afecte el trabajo activo de los equipos que dependen de ellos.

Lección 29 — Troubleshooting de pipelines fallidos

Los logs del Pod correspondiente a ese componente específico, para identificar el error exacto que causó el fallo dentro de ese paso particular.

El estado general 'Failed' del pipeline completo solo indica que algo salió mal en algún punto, sin explicar qué exactamente; los logs del Pod específico del componente que falló contienen el mensaje de error real y detallado necesario para entender la causa concreta del problema en ese paso particular.

Lección 30 — Proyecto final: plataforma Kubeflow completa lista para producción

Multi-tenancy con Profiles y RBAC bien configurado + Pipelines automatizados con recurring runs + Katib para tuning + KServe con InferenceServices en producción + linaje con MLMD + monitoreo activo.

Intenta mencionar al menos dos pilares: multi-tenancy con Profiles/RBAC, Pipelines automatizados, Katib para tuning o KServe con InferenceServices en producción.